

SALESTORM

Design Pattern Mapping

Team: Binary Minds **Members:** [Jumaanah Basheeth , Harini C , Manasa Devi] **Date:** 5 October 2026

1. Purpose

Each pattern below is used for a specific problem in the flash-sale design, and each one lists the trade-off it introduces. Class names match the class diagrams in 03_LLD.

2. Pattern Summary

2.1 Object-oriented patterns

Pattern	Where in SALESTORM	Problem solved	Trade-off introduced
State	ReservationState (Reserved, PaymentPending, Confirmed, Sold, Released) and OrderState (Created ... Delivered, Cancelled)	Status checks and illegal transitions scattered as if/else code. Each state class says which transitions it allows, and anything else is rejected (409 INVALID_STATE).	More classes. States must be mapped from the DB status string. It guards in-memory logic only, so the DB conditional update still protects concurrency.
Strategy	StockGate (RedisStockGate, DbOnlyStockGate), PricingStrategy (Standard, FlashSale, Coupon), delivery-partner selection	Swap an algorithm without touching the caller. DB-only gate takes over if Redis is down. A new pricing rule is a new class.	Extra indirection. Needs selection logic (config or feature flag) and tests per strategy.
Factory	PaymentProviderFactory.create(providerCode)	Callers never know concrete provider classes or how they are wired (and wrapped with resilience).	A new provider needs a factory entry. Use a registry or DI container so it is a config line, not an edit to core code.
Adapter	GatewayAAdapter, GatewayBAdapter, PartnerAAdapter, PartnerBAdapter, notification provider adapters	External APIs differ and change. Adapters translate them to our PaymentProvider and DeliveryPartner interfaces, so vendor changes stay in one class.	Mapping code to maintain. A common interface can hide vendor-specific features.
Circuit Breaker (as a Decorator)	ResilientPaymentProvider wraps any PaymentProvider with timeout, retry with backoff and CircuitBreaker	A failing gateway must not exhaust threads or cascade. Fail fast while open, probe while half-open.	Thresholds need tuning. An open circuit rejects some valid requests. Retries are only safe because calls are idempotent.
Observer	OrderEventBus with NotificationListener, ShipmentListener, InventoryConfirmListener. Across services this is publish/subscribe on Kafka	The Order Service should not call every downstream service. New consumers are added without changing it.	Eventual consistency and harder debugging. At-least-once delivery means every consumer must be idempotent.
Facade	CheckoutFacade.startCheckout()	One simple entry point hides the sequence: validate reservation, price, mark payment pending, pay.	Can grow into a "god class". It must only delegate. Cross-service failures are handled by events, not by a distributed transaction.

Pattern	Where in SALESTORM	Problem solved	Trade-off introduced
Repository	InventoryRepository, ReservationRepository, PaymentRepository, OrderRepository	Persistence is hidden behind interfaces. The atomic conditional SQL lives inside the repository, and services stay database-agnostic and easy to fake in tests.	Another layer. The conditional-update semantics can leak through the abstraction, so document them on the interface.

2.2 Distributed-system patterns

Pattern	Where in SALESTORM	Problem solved	Trade-off introduced
Transactional Outbox	OutboxEventPublisher, OUTBOX_EVENT table, Outbox Relay	A state change and its event must not diverge. Both are written in one local transaction, and the relay publishes later.	Extra table and relay process, and slight publish delay.
Idempotent Receiver	IdempotencyFilter, processed_message table, unique constraints	Duplicate requests and redelivered messages produce the effect exactly once.	Key storage and lookups on the hot path. Relies on clients reusing the same key on retry.
Saga (choreography with compensation)	Reserve, then pay, then create order. Compensations: release reservation, refund payment	No distributed transaction across services. Each step has an undo path.	The overall flow is harder to see. Needs reconciliation and monitoring to catch stuck sagas.

3. How the Key Patterns Work Here

3.1 State: reservation and order lifecycles

- Reservation holds a ReservationState. Calling confirm() on a ReleasedState throws an invalid-transition error, which the API maps to **409 INVALID_STATE**.
- Allowed paths: Reserved to PaymentPending to Confirmed to Sold, plus Reserved or PaymentPending to Released.
- Repeating a release is harmless: UPDATE ... WHERE status = RESERVED changes 0 rows the second time, so the operation is idempotent.

3.2 Strategy and Factory: choosing behaviour without if/else

- **Stock Gate:** RedisStockGate is the fast default. If Redis is unavailable, the system switches to DbOnlyStockGate (slower but still correct) without any change to ReservationService.
- **Pricing Strategy:** CheckoutFacade asks the strategy for the price. Flash-sale and coupon rules live in separate classes.
- **Payment Provider Factory:** returns the right provider already wrapped with timeout, retry and circuit breaker.

3.3 Adapter and Circuit Breaker (Decorator): protecting against external failure

- Every gateway implements the same PaymentProvider interface (charge, queryStatus, refund), so PaymentService never sees vendor details.
- ResilientPaymentProvider implements the same interface and wraps another provider, so it is substitutable (Liskov) and can be stacked on any adapter.
- Circuit states (example thresholds): **CLOSED** normal, **OPEN** after about 50% failures over 20 calls (fail fast for 30 s), **HALF_OPEN** lets 3 trial calls through, then back to CLOSED or OPEN.
- When the circuit is open the customer gets 503 with Retry-After, and the reservation stays valid until it expires.

3.4 Observer: reacting to order events

- After the order is saved, OrderConfirmed is published once. Inventory marks the reservation SOLD, Shipment creates the shipment, and Notification messages the customer. They are independent subscribers.
- The Order Service does not know who listens, so adding a loyalty-points service later changes nothing in the Order Service.

3.5 Facade and Repository

- CheckoutFacade is the only class the controller calls for checkout. It stays stateless and delegates to the inventory client, pricing strategy and payment client.
- InventoryRepository.decrementIfAvailable() contains the atomic UPDATE ... WHERE available_quantity > 0. Business code only sees a boolean result.

3.6 Outbox, Idempotent Receiver and Saga

- **Outbox:** the payment result and the PaymentSucceeded event are saved in one transaction, so a crash can never leave a paid customer with no event.
- **Idempotent Receiver:** duplicate requests and redelivered events are recognised and ignored.
- **Saga:** failures trigger compensations (release the reservation, refund the payment) instead of a cross-service rollback.

4. Extensibility: Adding New Behaviour with Minimal Change

Change request	What we add	What stays unchanged
New payment provider (for example Gateway C)	One class GatewayCAAdapter implements PaymentProvider, plus one registry or config entry GATEWAY_C. It automatically gets the resilient wrapper.	PaymentService, CheckoutFacade, Payment entity, database schema (the provider code is already stored), API.
New pricing strategy (for example early-bird or loyalty discount)	One class EarlyBirdPricing implements PricingStrategy, and a pricing type on the sale configuration.	CheckoutFacade, existing strategies, Inventory and Order services.
New delivery partner (for example Partner C)	One class PartnerCAAdapter implements DeliveryPartner, and one routing rule in the partner selector (Strategy).	Order Service, ShipmentListener, order state model, events.

Example: adding a payment provider

```
public interface PaymentProvider {
    ChargeResult charge(ChargeRequest req);
    ChargeResult queryStatus(String transactionRef);
    RefundResult refund(String transactionRef, Money amount);
}

// the only new file for Gateway C
public class GatewayCAAdapter implements PaymentProvider {
    // translate our requests to Gateway C's API and map its responses back
}

// registry entry (configuration, not core logic)
providers.put("GATEWAY_C", new ResilientPaymentProvider(new GatewayCAAdapter(), breaker, retry,
timeout));
```

A shared contract test suite for PaymentProvider (success, decline, timeout, duplicate charge with the same transaction reference) must pass for every new adapter before it is enabled.

5. Patterns Deliberately Not Used

- **Singleton for shared state:** services are stateless and horizontally scaled, so shared in-memory state would break under multiple instances. Dependencies are injected instead.
- **Distributed two-phase commit:** it blocks under failure and does not scale. We use outbox, idempotency and saga compensation instead.
- **One large event-driven orchestrator:** the checkout path stays synchronous where the customer needs an immediate answer, and events are used only after payment.